Episode 1 - Fundamental

1. How the Internet Works:

- History of Web (`Web 1.0` to `Web 3.0`).
- How `computer` communicate with each other.
- How computer send `data` all over the world.
- What is Domain Name, `IP` & `MAC` Addresses and Routing.
- How `ISP` and `DNS` work together to deliver data.

2. Client-Server Architecture:

- What is `Client-Server` Model.
- Difference between `Client` (browser) and `Server` (the computer hosting your website).
- How `HTTP request` and response cycle works (how browser talk to server).
- What happens when you visit a website.
- Difference between `Front-end` and `Back-end` (Front-end vs Back-end).
- What are `Static Websites` and `Dynamic Websites.`
- What is `web hosting` and how it works.

3. Internet Protocols:

- What is `TCP protocol` and why is widely used
- How Connection is established using `TCP` (3 Way handshake)
- What is `UDP` and why its used for fast communication
- How `UPD` establishes connection
- Difference between `TCP` and `UPD`

4. Understanding HTTP & HTTPS

- What is `HTTP` and its different version
- HTTP status code for responses
- What is `HTTPS` and why its better than `HTTP`
- How `HTTPS` provides a secure connection
- What is `SSL/TLS Encryption`
- What are `Proxy` and `Reverse Proxy`
- How `VPN` works and helps accessing restricted content

5. Preparing Your Machine:

- Installing & Setting up `VS Code`
- Installing helpful extensions
- Setting up your browser for `development`
- What are `file` and `folders` and how to create them
- Testing our environment via serving a webpage - " `Namaste Duniya` "

5. Semantic HTML & Browser Rendering:

- How Browsers Render Pages – HTML parsing → `CSSOM` → `Render Tree` → Paint.
- Understanding `Reflow` & `Repaint` – Why `layout` shifts happen.
- Semantic Tags – `header` , `main` , `section` , `article` , `aside` , `footer` explained architecturally.
- SEO Impact of `Semantic HTML` – Why structure improves search ranking.
- Accessibility Basics – Intro to `ARIA` roles and inclusive design.
- Content Structure – Headings `hierarchy` for clean document flow.
- Lists – `Ordered` & `unordered` for structured content.
- `Responsive Images` – Basic introduction to `image optimization` .
- Links & Navigation Flow – `Internal` vs `external linking` strategies.
- Forms Introduction – Structure of form, `input types` , `validation basics` .

6. CSS Core Fundamentals:

- CSS Syntax & Selectors – `Element` , `class` , `id` , `grouping selectors` , `parent selector` .
- `Specificity` & `Cascade` – Why some styles override others.
- `Inheritance` & Computed Styles – How `CSS` flows across elements.
- Box Model Deep Understanding – Content, `padding` , `border` , `margin` .
- Units – `px` vs `rem` vs `em` vs `%` , `vw` , `vh` .
- Modern CSS Functions – `clamp()` , `min()` , `max()` for `fluid design` .
- CSS Variables – Using : `root` for global theming.
- Design System
- Design Tokens Concept – Building `reusable spacing` & color systems.

7. Working With SASS (SASSY):

- What is SASS? `Variables` ,`Nesting` ,`Mixins` ,`Functions and Operators` .
- Setting up environment for `SCSS` .
- SCSS or SASS? and Setting Up `SCSS` .
- Working with SASS :- `Variables` , `Nesting` ,`Partials and Imports` ,`Mixins` ,`Inheritance/Extends` , `Functions` ,`Operators` .
- Advanced Concepts :- `Control Directives` , `Color Functions`

8. CSS Layout Mastery:

- Flexbox Deep Dive – `Axis control`, `alignment`, `distribution mastery` .
- Common Layout Patterns – `Navbar`, `card grid`, `center alignment` tricks.
- CSS Grid – `Grid template` areas & `2D layout control`
- Combining `Grid` & `Flex` – Real-world `layout architecture` .
- Positioning – `relative`, `absolute`, `fixed`, `sticky` with practical examples.
- Stacking Context – Understanding `z-index` properly.
- Container Queries – `Modern responsive` component design.

9. CSS Responsive Design:

- `Mobile-First Strategy` – Why starting small wins.
- Media Queries – `responsive` for different display strategy.
- `Breakpoint` Planning – Structured `scaling` approach.
- Responsive Images – `object-fit` ,`performance` awareness.
- Fluid Typography – `clamp()` for dynamic text sizing.
- Spacing Rhythm – Consistent `vertical` and `horizontal spacing` systems.
- CSS Frameworks - Bootstrap, `Tailwind CSS` .

10. CSS Architecture & Project Structure [legacy]:

- `BEM` Naming Convention – Avoiding messy class systems.
- Component-Based CSS – Designing reusable style blocks.
- `Utility Classes` vs `Component Classes` – Pros & cons.
- Real-World Folder Structure – `css/`, `components/` , `layout/` , `utilities/` .
- Debugging with DevTools – `Inspecting` overflow & `layout` shifts.

11. CSS Frameworks - TailwindCSS & Bootstrap

- Introduction to `CSS` frameworks and their role in `modern frontend` development.
- Understanding `utility-first` vs `component-based styling` approaches.
- Overview of `TailwindCSS` and why it is popular among developers.
- Setting up `TailwindCSS` in a project.
- Working with `Tailwind utility classes` for `layout` , spacing, colors, and typography.
- Building responsive `layouts` using `TailwindCSS` .
- Introduction to `Bootstrap` and its `component-based design` system.
- Using the `Bootstrap grid system` for `responsive layouts` .
- Working with common `Bootstrap` components like navbar, cards, and buttons.
- Customizing `Bootstrap` styles for `real-world projects` .
- Comparing `TailwindCSS` and `Bootstrap` .
- When to use `TailwindCSS` vs `Bootstrap` in practical development.

12. CSS Animations & Motion Design:

- CSS Transitions – `Timing functions` & `duration psychology` .
- Transform – `translate`, `scale`, `rotate` for `interactive` UI.
- `3D Transforms` Introduction – Adding depth perception.
- Keyframe Animations – Creating `smooth motion` systems.
- Performance Considerations – When NOT to animate.

13. Introduction to JavaScript

- What is `JavaScript` & Why It Powers the Web.
- Where `JavaScript` Runs – `Browser` & `Node.js` overview.
- Linking `JS` with `HTML` using `script` tag.
- Using Console for `Debugging` & `Testing` .
- Variables – `var` , `let` , `const` with scope understanding.
- Naming Conventions & `Clean Code` Practices.
- Data Types – `number` , `string` , `boolean` , `null` , `undefined` . [`Primitive` & `Non-Primitive`]
- Mini Practice Programs – Calculator, Greeting system, Input handling via prompt.

14. Document Object Model Manipulation

- Introduction to DOM in JavaScript

- Understanding DOM Structure and Tree - [`nodes` , `elements` , `document`]
- Fetching Elements in DOM - [`document.getElementById` , `document.getElementsByTagName` , `document.getElementsByClassName,` `document.querySelectorAll` , `document.querySelector`]
- DOM Tree Traversal - [`parentNode` , `childNodes` , `firstChild` , `nextSibling`]
- Manipulating DOM Element in JavaScript - [`innerHTML` , `textContent` , `setAttribute` , `getAttribute` , `style property` , `classList`]
- Create and Removing DOM Elements - [`createElement()` , `appendChild()` , `insertBefore()` , `removeChild()`]

15. Event Handeling in JavaScript

- Event Handling in JavaScript - [`addEventListner()` , `event bubbling` , `event.target`]
- Understanding Scroll Events, Mouse Events, Key Events and Strict Mode.
- Working with Forms and Input Elements [`Accessing Form Data` , `Validating Forms` , `preventDefault()` , `onsubmit` , `onchange`]
- Working with Classes ****Adding, Removing , Toggling (classList methods)
- Browser Events - [`DOMContentLoaded` , `load` , `resize` , `scroll`]

16. Using Browser Functionalities in JavaScript

- Browser Object Model - [`window` , `navigator` , `history` , `location` , `document`]
- Window Object - [`window.location` , `window.history`]
- Working with Storage - [`Local Storage` , `Session Storage` , `Cookies`]
- Web APIs in DOM - [`Fetch API`]

17. Asynchronous Programming JavaScript

- Introduction to Asynchrony in JavaScript.
- Introduction to `callbacks` and Problems in Callbacks
- Understanding `promises - pending` , `resolved` , `rejected`
- How to prevent callback hell using `async` & `await` .
- `setInterval` & `setTimeout` in JavaScript

18. Operators & Type System

- Arithmetic Operators – `+` , `-` , `*` , `/` , `%` , `++` , `--` .
- Comparison Operators `==` vs `===` .
- Logical Operators – `AND` , `OR` , `NOT` usage patterns.
- `Assignment Operators` – Practical usage.
- `typeof` Operator – Identifying data types dynamically.
- `Truthy` & `Falsy` Values – Real-world `conditional traps` .
- Type Coercion – `Implicit` vs `explicit` conversion.

19. Conditionals & Loops

- `if` , `else-if` , else branching logic.
- `switch statement` for structured decision-making.
- `for loop` iteration patterns.
- `while` & `do-while` usage cases.
- `break` & `continue` control flow statements.
- Practice – `Prime number check` , `multiplication table` , `pattern printing` .

20. Functions

- `Function Declaration` vs `Function Expression` .
- `Parameters` & `Arguments` deeply explained.
- Return `Values` & `Scope` .
- Arrow Functions (`ES6+`).
- Writing `Reusable Logic` Blocks.
- Practice – `Grade calculator` , even/odd checker, `reusable` utility functions.
- `Recursion`

21. Arrays:

- Creating Arrays – What `arrays` are and when to use them.
- Accessing Elements – Indexing basics.
- Array Operations – `push` , `pop` , `shift` , `unshift` .
- `length Property` – Counting items in an array.
- Basic `Iteration` – Looping with `for` / `while` .
- Practice – Todo list in `array` , `max` / `min` finder.
- `Multidimension Array`

22. Array Methods:

- `map()` – Transforming `arrays` .
- `filter()` – Selecting items based on conditions.

- `reduce()` – Accumulating values into a single output.
- `forEach()` – Iteration without returning a new `array`.
- `some()` / `every()` – Boolean checks on arrays.
- `slice()` / `splice()` – Copying vs modifying arrays.
- `sort()` - Sorts the array in place and returns it.
- `flat()` - Flattens `nested arrays` into one level.
- Introduce `functional thinking` lightly.
- Practice – `Filter students by marks`, transform user list, calculate total price.

23. Objects:

- What is an `Object`? – Real-world modeling with `key-value` data.
- `Key-Value` Pairs – Understanding `properties` and `values`.
- Accessing Properties – `Dot notation` vs `bracket notation`.
- Adding / Deleting Properties – `Mutating objects` safely.
- `Nested Objects` – Deep data structures.
- Looping Through Keys – for...in and `Object.keys()`.
- Practice – User profile system, cart object.

24 Object Methods:

- `Object.entries()` – Converts an object into an array of `[key, value]` pairs.
- `Object.assign()` – Copies properties from source objects into a target object.
- `Object.freeze()` – Makes an object immutable (no add, delete, or update).
- `Object.fromEntries()` – Converts an array of `[key, value]` pairs back into an object.
- `Object.is()` – Compares two values with edge-case safe equality (useful for `NaN`).
- `Object.keys()` – Returns an array of all keys in an object.
- `Object.seal()` – Prevents adding or deleting properties (values can still change).
- `Object.values()` – Returns an array of all values in an object.

25. TypeScript Essentials:

- What is `TypeScript` and why it exists.
- Thinking in `TypeScript` – Types-first mindset.
- Installing and Setting Up `TypeScript`.
- Basic Types – `string`, `number`, `boolean`, `any`, `unknown`.
- Interfaces vs Type `Aliases` – When to use what.
- `Union` and `Intersection Types` – Modeling flexible data.
- `Generic Functions` – Reusable logic with type safety.
- Generic Interfaces and Constraints – Safe boundaries.
- TypeScript Tooling
- `tsconfig.json` Explained.
- TypeScript Compiler (`tsc`).
- `Linting` and `Formatting` with `TypeScript`.

Some Additional Topics

- `Throttling` and `Debouncing` uses in JavaScript
- JSON Handeling and JavaScript - [`JSON.parse()`, `JSON.stringify()`]

Episode 2 – React, Frontend Architecture & Next.js

1. Introduction to React – Why Modern UI Needs It

- Why React Exists – Problems with manual `DOM` manipulation in `large-scale` apps.
- `Declarative` vs `Imperative UI` – Understanding mindset shift in `UI` engineering.
- What is SPA (Single Page Application) – Why page doesn't reload in modern apps.
- `Real DOM` vs `Virtual DOM` – How React optimizes `rendering performance`.
- Setting Up `React` with `Vite` – Installing project, `folder understanding`.
- Project Structure Breakdown – `src`, `components`, `assets`, `public folder` explained.
- `JSX Syntax` Rules – `Embedding JavaScript` inside HTML-like syntax.
- `Component-Based Architecture` – Breaking UI into reusable logical units.

2. React Components & Props

- Functional Components – `Writing reusable UI` functions.
- `Understanding Props` – Passing data between components.
- `Dynamic Rendering` – Rendering UI based on props data.
- `Rendering Lists` – Using `map()` to generate repeated UI blocks.
- Keys in React – Why `unique keys` are critical for `performance`.

- Creating `Reusable` Card Component – Mini practical implementation.

3. State & Re-rendering Logic

- What is `State`? – Data that changes over time.
- `useState` Hook – Managing local component state.
- How React `Re-renders` – Understanding reconciliation process.
- Batching State Updates – `Performance optimization` internally.
- `Derived State` Concept – Avoiding unnecessary state variables.
- Practical Build – `Interactive counter`, toggle switch UI.

4. React Lifecycle Methods

- `Class Components Lifecycle` – Component lifecycle flow in class components.
- `React's Lifecycle Methods` – componentDidMount, componentDidUpdate, componentWillUnmount.
- `useEffect` Hook – Side effects in functional components.
- `Data Fetching`, `Cleanup` & `DOM Manipulation` – Practical `patterns` and `best practices`.

5. Event Handling & Conditional Rendering

- Handling `Click Events` in React.
- `Controlled Inputs` – Managing `form inputs` via `state`.
- Conditional `Rendering Patterns` – `&&` `operator`, `ternary operator`, `early return`.
- `Dynamic UI Rendering` – Showing/hiding components.
- Practical Build – Dynamic form with `validation logic`.

6. Component Architecture Principles

- `Single Responsibility Principle` – One component, one responsibility.
- Smart vs Dumb Components – `Separation` of concerns in `UI`.
- Lifting State Up – Sharing data between `sibling components`.
- `Prop Drilling Problem` – Why deep passing becomes messy.
- `Component Composition` – Building `scalable` layout systems.

7. Useful Hooks in React

- Understanding `React Hooks`
- Rules of `hooks`.
- Commonly Used Hooks: `useState`, `useEffect`, `useContext`, `useRef`, `useCallback`, `useMemo`.

8. useEffect Deep Dive

- What `useEffect` Really Does – `Synchronizing` with external systems.
- `Dependency Array` Behavior – Why infinite loops happen.
- Cleanup Functions – Preventing `memory leaks`.
- Data Fetching `Pattern` with `useEffect`.
- Practical Example – `Fetching API` data on component mount.

9. Custom Hooks & Reusable Logic

- Why `Custom Hooks` Exist – Extracting shared logic.
- Writing a `useFetch` Hook – Reusable API fetching logic.
- Separation of `Business Logic` from `UI`.
- Improving `Code Maintainability` with `Hooks`.

10. Advanced Reusability Patterns

- `Higher Order Components (HOCs)` – Reusing behavior by wrapping components.
- `Render Props Pattern` – Sharing logic through a function prop.
- `Compound Components` – Designing flexible component APIs with shared internal state.

11. Routing & Application Structure

- Introduction to `React Router` – Client-side navigation system.
- Defining `Routes` & `Nested Routes`.
- `Dynamic Routing` & URL Parameters.
- useNavigate Hook – Programmatic navigation.
- `Layout` Components – Shared header/sidebar architecture.
- Folder Structure for Scalable Projects – `components/`, `hooks/`, `pages/`, `services/`, `utils/`.

12. Server State & API Integration

- `Fetch API Integration` in React.
- Handling Loading & `Error States` Gracefully.
- Separation of `Server State` vs `Client State`.
- Introduction to `TanStack Query` – Smart caching & refetching.
- `useQuery` Basics – Data fetching pattern.
- `Mutations` – Updating server data properly.

13. Global State Management

- `Context API` - Managing app-wide state.
- `Context Providers` & Consumers – Sharing state cleanly across `deep component trees`.
- `Zustand` - Lightweight alternative to heavy Redux.
- When to Use `Global State` vs `Local State`.
- Avoiding `Overengineering State Logic`.

14. State Management Using Redux.

- Introduction to `Redux`, What is redux?, When and Why use redux?
- Understand Principles of `Redux` and `Redux Flow`.
- Understanding `State Management` in React using `Redux`.
- Why Use `State Management` Libraries?
- Why Redux need reducers to be `pure functions`.
- Redux Basics: `Actions`, `Reducers`, `Store`, `Currying`, `Middleware`, `Async Actions: Thunk`
- Connecting Redux to React Components with `react-redux`.
- Introduction to `Redux Toolkit.`
- Alternatives: Recoil, Zustand, or MobX.

15. Advanced Forms & Validation

- `React Hook Form` – Efficient form handling.
- `Controlled` vs `Uncontrolled` Forms.
- `Zod Schema Validation` – Type-safe validation strategy.
- Error Display Patterns.

16. Performance Optimization in React

- `React.memo` – Preventing unnecessary re-renders.
- `useMemo` – Memoizing expensive calculations.
- `useCallback` – Stable function references.
- `Code Splitting` with `React.lazy` & `Suspense`.
- `Lighthouse` Performance `Testing`.
- Debugging `Re-renders` via `DevTools`.

17. Understanding Full Stack Frameworks

- What are `Full Stack Frameworks` and how they help build both `frontend` and `backend` in a `single system`.
- Overview of `modern web frameworks` in 2026 and how they shape modern application development.

18. Getting Started with Next.js

- Why `Next.js` is widely used for building modern web applications.
- `Next.js` vs `React` — understanding the difference between a `framework` and a UI library.
- Setting up a new `Next.js` project.
- Understanding `File-Based Routing`.
- Working with `Layouts` in `Next.js`.

19. Rendering Strategies in Next.js

- `Server-Side Rendering` (`SSR`) — generating pages on the server.
- `Static Site Generation` (`SSG`) — pre-building pages for performance.
- `Incremental Static Regeneration` (`ISR`) — updating static pages without rebuilding the entire app.

20. Building APIs with Next.js

- Creating backend APIs using `Next.js`, `API` routes.
- Implementing HTTP methods — `GET`, `POST`, `PUT`, `PATCH`, `DELETE`.
- Connecting `APIs` with databases.
- Structuring `responses` and handling errors.

21. Working with Server Actions

- Understanding `Server Actions` and how they differ from `API routes`.

- Using the `use server` directive.
- Implementing `Server Actions` for handling `server-side logic`.

Episode 3 – Node.js, MongoDB & Backend Architecture

1. Starting with Node.js - The Beginning

- Introduction to `Node.js` and Getting Our Tools - `Node.js LTS` , `Postman` , `Editor`
- Setting up the `Tools` for `our Environments`
- Running `script` with `nodejs` - `"Namaste Duniya"`
- `NPM` Basics | Installing `Packages`.
- Creating and Managing `package.json`.

2. Creating Server - Writing Our First Server

- What is Server and how it works?
- Setting Up Our First `Node.js Server` using `HTTP`
- `Serving A Response` to the Browser and Understanding Responses.
- `Routing` in `HTTP Servers`.
- Understanding Status Code - `1XX` , `2XX` , `3XX` ,`404 - Not Found` , `200 - success` , `500 - Internal Server error` , `422 - Invalid Input` , `403 - the client does not have access rights to the content` , etc.
- Installing `Nodemon` for Automatic Server Restarts.

3. Some talk on Different Architectures

- Different Architectures in backend like `MVC` and `SOA`.
- Understanding MVC Architecture `Model` , `View` ,`Control`.
- MVC in the context of `REST APIs`.

4. Web Framework - Express.js

- what is `Express.js` and why to use it.
- Setting Up `Express Server` .
- Returning Response from the `server`.
- Using `Query Parameters` and `URL` Parameters.
- HTTP Request - `Some Important part of requests` , Different Types of Requests - `Get` , `Post` , `PUT` , `Patch` , `Delete`.
- Serving Static Files with `express.static()` .

5. Template Engine - EJS

- What is `Template Engine` and What is the use of `Template Engine`.
- Template Engine Option - `Handlebars` , `EJS` , `Pug` , `jade` but We'll use `EJS` .
- Setting Up Template Engine - `Installed EJS template engine`.
- Rendering Our First Page using `EJS` and Some important syntax - `<%= %>` , `<% %>` , `<%- %>`.
- Loop statement, Conditional statement and Locals in views - `EJS`.
- Accessing the Static Files Inside `EJS` file.

6. Middleware in Express.js (one of my favorite)

- Understanding the `middleware` in express.
- Implementing `middleware` with express.
- Different types of middleware : `builtIn middleware` , `third-party middleware` ,`custom middleware` .
- Different level of middleware : `Application-Level` , `Router-Level` .
- Handeling Errors and Security with middleware : `Error-Handling` , `Helmet` , `CORS`.

7. Handling file with Express

- Understand `Multer` and its usecase?
- Uploading file with `multer`.
- Understanding `Memory` and `Disk` Storage.
- Accessing uploaded file `req.file`.
- Working with `express.static`.
- Using `Cloudinary` or `Imagekit` for Real-time media processing APIs and Digital Asset Management.

8. Beginning of Database Basics (Bohot km theory)

- Relational and non-relational Databases : `mongodb` & `mysql` .
- What is `MongoDB` ? Why Use It?
- Installing `Compass` and Understand `how to access DB using terminal` .

- Setting Up MongoDB `Locally` and in the `Cloud`.
- Understanding `Datatypes` `Collections` and `Documents`.
- Connecting MongoDB to Node.js with `Mongoose` .
- Database Relations - `One to One` , `One to Many OR Many to One` , `Many to Many` , `Polymorphic` .
- Handling Relationships with Mongoose (`populate`).

9. API Development(REST)

- What is a `REST API` ?
- Versioning in RESTful APIs - `/v1/`
- Using `Postman` for API Testing and developing - `Send Requests` , `Save Collections` , `Write Tests` .
- Understanding and Working With `Status code` , `2xx (Success)` , `4xx (Client Errors)` , `5xx (Server Errors)` .
- Validating API Inputs Using libraries like `express-validator` or `Sanitization` .
- Security Handling - Rate Limiting with `express-rate-limit` ,`XSS Attack` , `CSRF Attack` , `DOS Attack` .

10. Input Validation

- Why `validation` matters
- `express-validator` OR `Zod`
- `Sanitization`
- Proper error response format

11. Database Optimization for Fast response

- `Indexing` for Performance with MongoDB :- `Single-Field Indexes` , `Compound Indexes` , `Text Indexes` ,`Wildcard Indexes` .
- Best practice with Indexing `explain()` .
- Learning MongoDB `Aggregation` .
- Comparison Operators - [`$eq` , `$ne` , `$lt` , `$gt` , `$lte` , `$gte` , `$in` , `$nin`]
- Logical Operators - [`$not` , `$and` , `$or` and `$nor`]
- Array[`$pop`, `$pull`, `$push` and `$addToSet`]
- `Stages` in Aggregation pipeline :- `$match` , `$group` ,`$project` ,`$sort` ,`$lookup` .
- Creating Database on `Local` and `Atlas`
- Creating parallel pipeline with `$facet` .
- Learning MongoDB `Operators` .
- Understanding Different types of Operators :- `Comparison` ,`Regex` ,`Update` ,`Aggregation` .

12. Logging & Monitoring

- Why is `Logging` Important?
- Setting Up Logging with Libraries `winstone` ,`Pino` ,`Morgan` .
- Different mode of `morgan` ,**`dev`** ,**`short`** ,**`tiny`** .
- `Error Handling` and Logging.

13. Production Wala Project Structure and Configuration

- Understanding the Basic Structure of application.
- Learning File Naming Conventions, Git Configuration,
- Understanding Important Folders :- `src/` ,`config/` ,`routes/` ,`utils/` .
- Role of `package.json` , `ENV` and `.gitignore` .
- Production Environment - `PM2` , `Error & Response Handling Configuration` , `CORS Configuration` , `async-handler.js` .
- Using and Configuring `ESLint` and `Prettier` for code formatting.
- Testing APIs using `Postman` .

14. Authentication and Authorization

- Difference Between `Authentication` & `Authorization`
- Working with `Passwords` and `Authentication` - `Cookie Authentication` , `OAuth Authentication`
- Understanding `Session` and `Token Authentication` .
- Implementing JWT Authentication :- `jsonwebtoken` `JWT_SECRET` .
- Securing user password with `bcrypt` `hashing` `salt` .
- Role-Based Access Control (`RBAC`).
- Authenticating user with `Express middleware` .
- Understanding `Passport.js` and its usecase?
- Glancing through and Installing `Passport.js`
- Setting up Passport.js - `passport-local` , `local-strategy` , `google-OAuth`
- `express-sessions` and using passport for authentication.

15. Working Real time communication : `WebSockets` and `socket.io`

- Understanding `WebSockets` protocol for realtime applications?
- Learning `handshake` ,`Persistent connection` ,`Bidirectional communication` ,`HTTP polling` .
- Understanding difference between `WebSocket` Vs `Socket.io` .
- Working with `socket.io` for realtime applications.

- Understanding usage of `Rooms` in Socket.io.
- Understanding `Middleware` in Socket.io.

16. Working With Caching - Local and Redis

- What is `Caching` and How to cache data locally?
- What is `Redis` ?
- Why Use Redis for `Caching` ?
- Implementing `Redis Caching` in `Node.js` .
- Advanced Redis Features `TTL` , `Complex Data Structures` , `Pub/Sub` .

17. Error handling in express

- Basic Error Handling in Express `next()` .
- Catching Specific Errors `try` & `catch` .
- Creating `Util` Class for `Error Handling` .

18. Testing Tools

- Understanding Unit-Testing With `Jest` .
- `Cross Browser Testing` and Why Is It `Performed` ?
- What Is `Web Testing` ? and How to Test a Website.

Episode 4 – Generative AI Engineering

1. How LLMs Actually Work

- What are `Tokens` and why everything becomes tokens.
- What is `Context Window` and why prompts get cut.
- Sampling Basics – `temperature` and randomness control.
- Why `Hallucination` happens (it predicts, it doesn't "know").
- Difference between `Deterministic` vs `Probabilistic` outputs.

2. Calling LLM APIs Properly

- Message roles – `system` , `user` , `assistant` .
- `Token budgeting` – prompt size, response size, and overflow planning.
- `Cost awareness` – pricing per token and why bad prompts waste money.
- Rate limits – handling `429` , backoff and retries.

3. Prompt Engineering Fundamentals

- Write clear instructions (don't be vague).
- Few-shot examples to `force pattern learning` .
- Chain-of-thought: only `high-level awareness` (don't depend on hidden reasoning).
- `Output formatting instructions` (make the response predictable).

4. Structured Output in AI Systems

- Why `unstructured AI responses` are difficult to use in real systems.
- How structured responses make `AI outputs predictable` and programmable.
- Common applications such as resume `parsing` , `product metadata generation` , `moderation systems` , and `compatibility scoring engines` .

5. Generating JSON Responses from LLMs

- Using model-supported `JSON mode` for structured outputs.
- `Guiding models` with schema-based prompting.
- Basics of `tool/function calling` .
- `Validating` the `returned` structure before using it.
- Building an example where `AI returns structured profile` analysis data:

```
{
  "compatibility_score": 82,
  "strengths": [],
  "weaknesses": [],
  "suggested_openers": []
}
```

6. Schema Validation with `Zod`

- Defining a `validation` schema using `Zod`.
- Verifying `AI responses` against a `schema`.
- `Handling invalid` or mismatched responses.
- Implementing `retry strategies` for broken outputs.
- Principle: never trust `AI output` without validation.

7. Function Calling / Tool Calling

- What is `tool calling` and why it matters.
- When to use it (`deterministic actions`).
- `Deterministic` tool invocation.
- `Hybrid logic` (AI + code).
- We'll make AI decides which function to call: `improveBio()`, `generateOpeners()`, `moderateText()`.

8. Streaming Responses

- `Streaming tokens` from the model instead of waiting for full output.
- `Progressive rendering` in user `interfaces`.
- Improving `UX` in `AI-driven` applications.

9. Combining Streaming with Structured Data

- `Streaming explanation` or `conversational text`.
- Returning `structured JSON` data once generation finishes.
- Separating `UI responses` from `system-readable` outputs.

10. Error Handling in AI Applications

- Handling invalid or `malformed JSON` responses.
- Managing `partial responses` and `timeouts`.
- Implementing `retry mechanisms`.
- `Logging AI` requests and responses for `debugging`.

11. Understanding Embeddings

- What embeddings represent in `vector space`.
- How `similarity search` works conceptually.
- Document chunking strategies for `better retrieval`.

12. Integrating Vector Databases

- Storing embeddings for search and retrieval.
- Querying similar vectors efficiently.
- Using metadata filters during search.

13. Building a RAG Pipeline

- Basic `RAG` workflow:

    1. User `query`
    2. Retrieve `relevant chunks`
    3. `Inject into prompt`
    4. `Structured answer`

- Build: Document QA system returning structured JSON:

```
{
  "answer": "",
  "confidence": 0.92,
  "sources": []
}
```

14. Evaluating AI Systems

- `Detecting hallucinations`.
- Assigning confidence scores.
- Evaluating retrieval quality.
- Observability and monitoring basics.

15. Understanding AI Agents

- `Agent` vs single `LLM` call.
- Agent = `reasoning` + tool usage.
- When to use `multi-agent` systems.
- When NOT to use them.
- `Multi-agent` systems increase complexity.

16. Agent Design Patterns

- `Planner` → Executor workflow.
- `Researcher` → Writer pattern.
- `Critic` → Refiner loop.
- `Router agents` that decide which tool to call.

17. LangChain (Practical)

- Introduction to `LangChain`.
- Building `chains` and `prompt` templates.
- Using tools and memory modules.
- Working with `agents`.
- Overview of `LCEL` concepts.

18. Building a Simple Multi-Agent Workflow

- Example (Hinge direction):
  - Agent 1 – Profile Analyzer
  - Agent 2 – Bio Improver
  - Agent 3 – Conversation Starter Generator
  - Pipeline: Input → Analyze → Improve → Generate openers → Structured JSON output
- Example (ImageKit direction):
  - Agent 1 – Image Metadata Extractor
  - Agent 2 – SEO Optimizer
  - Agent 3 – Tag Categorizer
  - All outputs validated with schema

19. Multi-Agent Architecture Concerns

- `Increased latency`.
- `Higher operational cost`.
- `Debugging` difficulty across agents.
- Managing shared state between services.
- Importance of `logging` each step in the `pipeline`.
- Understanding that `multi-agent` systems are not always the best solution.

Episode 5 – Production Systems, Infrastructure & Deployment

1. `DevOps` Foundations

- Understanding the difference between `development` and `operations`.
- Basics of `CI` and `CD` for automated builds and deployments.
- Introduction to Infrastructure as Code (`IaC`).
- Why applications behave differently in `Production` vs `Localhost`.
- Understanding `Development`, `Staging`, and `Production` environments.
- Basic `Linux` usage and connecting to servers using `SSH`.
- Important Linux commands – `ls`, `cd`, `mkdir`, `nano`, `vim`, `ps`, `kill`, `top`.
- Managing backend applications in production using `PM2`.
- Running `Node.js` apps in background and `monitoring logs`.

2. Docker & Containerization

- Introduction to `Docker` and container-based deployments.
- Understanding `Images` vs `Containers`.
- Why containers improve `consistency` across environments.

- Containers vs traditional `Virtual Machines`.
- Writing a simple `Dockerfile` using `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `CMD`.
- Containerizing a backend service.
- Running multi-service apps using `Docker Compose`.
- Connecting services like `Backend`, `MongoDB`, and `Redis`.
- Optimizing builds using multi-stage Docker builds and `.dockerignore`.

3. Cloud Deployments

- Basics of `Cloud Computing`.
- Overview of major `cloud providers`.
- Understanding `scalability`, `high availability`, and f`ault tolerance`.
- Managing `infrastructure cost` and `performance`.

4. AWS EC2

- Introduction to `AWS` cloud platform.
- `Understanding regions` and availability zones.
- What `EC2` instances are and how they work.
- Launching and `configuring an EC2 instance`.
- Connecting to EC2 using `SSH`.
- Installing `Node.js` and `Docker` on the server.
- Deploying a `backend service` manually.

5. Security Groups

- What `Security Groups` are and why they matter.
- Managing `inbound` and `outbound` rules.
- Restricting `server` access for `security`.
- `Security best practices`.

6. Load Balancers

- What a `Load Balancer` does.
- Why `load balancing` improves `reliability`.
- Using AWS `Elastic Load Balancer`.
- Configuring `Application Load Balancer (ALB)`.
- Health checks for monitoring services.

7. AWS CloudFront

- Introduction to `CloudFront`.
- Using `CDN` for `faster content delivery`.
- `Caching` and `edge` locations.
- Serving `static assets` efficiently.

8.CI / CD Automation

- Understanding `Continuous Integration`.
- Creating workflows with `GitHub Actions`.
- `Running tests` and build pipelines.
- `Automating deployments`.
- Triggering deployments on `git push`.
- Deploying to servers via `SSH`.
- `Rebuilding Docker` containers automatically.

9. Scalability & Reliability

- `Vertical` vs `Horizontal` scaling.
- `Load balancing strategies`.
- Using `Redis` for caching.
- `Monitoring`, `logging`, and `observability` basics.

Episode 6 – Scaling, Reliability, Micro-Services & Web3

1. Microservice Foundations

- Understanding `Monolithic` vs `Microservices` architecture.
- When microservices make sense and when they add unnecessary complexity.

- Identifying proper service boundaries using domain-based thinking.
- The `Database-per-Service` principle in distributed systems.
- Communication approaches between services – synchronous (`REST`) and asynchronous (`Events`).

2. Building & Orchestrating Microservices

- Creating `independent services` with `separate codebases` and `configurations` .
- Running `multiple backend services` as separate `Node.js applications` .
- Understanding the `API Gateway` pattern for `centralized routing` and `authentication` .
- Forwarding requests from `client` → `gateway` → `internal services` .
- Handling `internal service` communication using `REST` calls.
- Retry `strategies` , `timeouts` , and `circuit breaker concepts` .
- Introduction to `event-driven architecture` .
- Using `RabbitMQ Pub/Sub` for event communication.
- Designing event `payloads` and `ensuring idempotency` .
- Understanding `eventual consistency` in `distributed systems` .

3. AWS Cloud-Native Deployment

- Using `Amazon ECR` to store and manage container images.
- Building `Docker images` locally and pushing them to `ECR` .
- Understanding `ECS` clusters and task definitions.
- Overview of `Fargate` vs `EC2` container execution.
- Routing traffic using `Application Load Balancer (ALB)` .
- Configuring target groups and service health checks.
- Understanding `VPC` , `Subnets` , and network security groups.
- Basic introduction to `IAM` roles and permissions.

4. Distributed Observability & Scaling

- Scaling backend services horizontally using `ECS` tasks.
- Understanding `stateless service architecture` .
- Using `health checks` to manage `service availability` .
- Monitoring services with `CloudWatch` .
- Observing `system resilience` by simulating service failures.
- `Tracking AI` usage and system events through `service logs` .

5. Scaling Concepts

- Difference between `Vertical` and `Horizontal` scaling.
- Using `load balancers` to distribute traffic.
- Importance of `stateless backend design` .
- Understanding `sticky sessions` and when to avoid them.

6. Caching in Production

- Using `Redis` for application caching.
- Understanding the `cache` invalidation challenge.
- Applying `TTL` strategies to manage cache lifetime.
- Preventing stale or outdated `cached data` .

7. Logging & Observability

- Using `Structured Logging` for backend systems.
- `Monitoring application logs` and system metrics.
- Understanding basic `observability principles` .
- Introduction to `alerting systems` .
- Logging important events like `API errors` , `authentication failures` , and `AI usage` .

8. Production Hardening

- Strengthening backend `security for production environments` .
- Implementing `rate limiting` for `APIs` .
- Proper `CORS` configuration.
- Understanding `DDoS` protection concepts.
- Using `secure HTTP` headers.

9. Final Production Deployment

- Deploying a complete system using `Docker` .
- Running services on `EC2` .
- Connecting a domain with `SSL` .

- Automating deployments using `CI` / `CD` pipelines.

10. Web3 Basics. (Introductory)

- Understanding the concept and potential of `Web3`.
- Fundamentals of `Blockchain` technology and how it powers Web3.
- Exploring Decentralized Applications (`DApps`) and their use cases.
- Introduction to `Smart Contracts` : How they work and their applications.
- Overview of `Cryptocurrencies` and their role in the `Web3` ecosystem.

Episode 7 - DSA with JavaScript

1. Conditional Statements

- Understanding `Conditional Statements`
- Types of Conditional Statements `if` , `if-else` , `if-else if` , `switch`
- Making decisions in a `program based on inputs` or `variables`.
- `Validating user data` or `input forms` .
- Creating `interactive menus` or options in applications.

2. Loops, Nested Loops, Pattern Programming

- Undertsanding the use of `Loops`.
- `for` loop.
- `while` loop.
- `do-while` loop.
- Understanding the Use of `Nested Loops` .
- Learning Pattern Programming - `Pyramid patterns` , `right-angled triangles` , and `inverted triangles` .
- Understanding Control Flow statement `break` and `continue`
- Learning how to set correct conditions to avoid getting stuck in `infinite loops` .
- Understand how to `optimize` nested loops for `better performance` and reduced time complexity.

3. Array

- Understanding the use of `Arrays`.
- Basic Manipulations - `insertion` , `deletion` , `updation`
- Accessing Elements in Arrays .
- Traversing Elements in Arrays .
- Array Algorithms - `Two Pointer Algorithm` , `Rotation Algorithms` , `Kadane's Algorithm` , etc.

4. Object-Oriented Programming (OOP) in JavaScript

- Understanding `Object-Oriented Programming`
- Learn how to define a `class` for creating objects.
- Understand how to instantiate `objects` from a class
- Learn how the `constructor()` function initializes an object when it's created.
- Understand how `this` refers to the current object in the context.
- Use `this` to access properties and methods within the same object.

5. Strings in JavaScript

- Understanding Strings in JavaScript
- Learning String Manipulation Methods - `concat()` , `slice()` , `substring()` , `replace()` , `replaceAll()`
- Learning String Search and Check Operations - `indexOf()` , `lastIndexOf()` , `includes()` , `startsWith()` , `endsWith()`
- Learning String Transformations - `toUpperCase()` , `toLowerCase()` , `trim()`
- Learning String Splitting and Joining: - `split()` , `join()`
- Embed variables and expressions in strings using backticks `()`
- Learning Escape Characters - `\n` , `\t` , `\'`
- Algorithms on Strings - `Reverse a String` , `Check for Palindrome` , `Find Longest Common Prefix` , `Character Frequency Count` , `Anagram Check`

6. Time and Space Complexity

- Understanding `Time Complexity`
- Understanding the `Big-O` Notation.
- Constant Time – `O(1)`
- Logarithmic Time – `O(log n)`
- Linear Time – `O(n)`
- Linearithmic Time – `O(n log n)`
- Quadratic Time – `O(n²)`

- Exponential Time – `O(2ⁿ)`
- Factorial Time – `O(n!)`
- Key Factors That Affect Complexity - `Algorithm Design` , `Data Structure Choice` , `Problem Constraints`
- Tips to Reduce Time Complexity - `Avoid Nested Loops` , `Efficient Data Structures` , `Optimize Recursion` , `Divide and Conquer`
- Understanding what is Recursion and its use case

7. Math Problems and Algorithms

- Understanding `Mathematical Operations` and Their Applications
- Mathematical operations like `(pow)` `(sqrt)` and greatest common divisor (HCF) are essential in various problem-solving scenarios.

8. Advanced Problems on Array

- Understanding `Advanced Array Concepts`
- Learning `two-pointer` approach ,
- Learning `prefix sums`
- Solving `complex problems` `efficiently` .
- `Multi-Dimensional` Arrays in JavaScript
- Working with `Multi-Dimensional Arrays`
- Key `Operations` on Multi-Dimensional Arrays
- Algorithms Using `Multi-Dimensional Arrays`
- `Multi-Dimensional Arrays` in Real-World Scenarios

9. Sorting Algorithms ,Time complexity and their application

- Learning `Selection Sort`
- Learning `Insertion Sort`
- Learning `Merge Sort`
- Learning `Quick Sort`
- Learning `Cyclic Sort`

10. Binary Search and Its Algorithms

- Binary Search on `Sorted Arrays`
- Variations of `Binary Search`
- Binary Search on `Infinite Arrays`
- Binary Search in `Rotated Sorted Array`
- Binary Search on `2D Matrix`
- `Real-World` Use Cases of `Binary Search`

11. Hashing (Set and Map) in JavaScript

- Understanding Hashing in JavaScript - `s**et` , `map` *
- Working with `Set` in JavaScript
- Methods in Set - `add(value)` , `delete(value)` , `has(value)` , `clear()` , `size`
- Working with `Map` in JavaScript
- Methods in Map - `set(key, value)` ,`get(key)` , `delete(key)` , `has(key)` , `clear()` , `size`
- Learning Algorithms Using `Set` & `map`

12. Linked List in JavaScript

- Understanding Linked List - `Data` , `Pointer`
- `Singly Linked List.`
- `Doubly Linked List.`
- `Circular Linked List.`
- Creating a Node in Linked List:
- Building a `Linked List` :
- `Traversing a Linked List` :
- Operations on Linked Lists - `Insertion` , `Deletion` , `Searching`
- Algorithms Using `Linked Lists`

13. Queue in JavaScript

- Implementation of `Queue` by `Linked List` and `Array`
- Working with Queues - `Basic Queue` , `Circular Queue`
- Operations on Queues - `Enqueue` , `Dequeue` , `Peek` , `IsEmpty` , `Size`
- Algorithms Using `Queues`
- Applications of `Queues`

14. Stack in JavaScript

- Understanding `Stacks` in `javaScript`

- Implementation of `Stack` by `Linked List` and `Array`
- Working with `Stacks`
- Operations on Stacks - `Push` , `Pop` , `Peek` , `IsEmpty` , `Size`
- Algorithms Using `Stacks`
- Applications of `Stacks`

15. Advanced Problems on Recursion and Backtracking

- Understanding `Advanced Recursion` and `Backtracking`
- Key Problems and Algorithms like `N-Queens Problem`,`Sudoku Solver`,`Subset Sum`,`Word Search`
- `Optimizing Recursive Solutions` with `Backtracking`
- Challenges with `Recursion` and `Backtracking`
- `Applications` of `Recursion` and `Backtracking`

16. Tree

- Understanding Binary Trees
- Types of Binary Trees - `Full Binary Tree` , `Complete Binary Tree` , `Perfect Binary Tree`
- Key Terminology in Binary Trees - `Node` , `Root` , `Leaf` , `Height of a Tree` , `Depth of a Node` , `Level of a Node`
- Binary Tree Operations - `Insertion` , `Deletion` , `Traversal` , `Searching`
- Binary Tree Algorithms - `Height` , `Diameter` , `LCA` , `Symmetry Check`
- Applications of Binary Trees

17. Binary Search Tree (BST):

- Understanding `Binary Search Tree`
- Properties of `Binary Search Tree`
- BST Operations
  - Binary Search Tree `Algorithms`
  - Applications of `Binary Search Tree`
  - Advantages of `Binary Search Tree`

Episode 8 - Aptitude and Reasoning

Classic Chapters

1. Percentage

- Learn `tips and tricks` for percentages.
- Solve `basic`, `medium`, and `advanced` questions.
- Practice `MCQs` to master percentages.

2. Profit and Loss

- Concepts of Profit and loss
- Relationship between `cost price`, `selling price`, and `mark-up price`.
- Solve practical scenarios involving `discounts`, `successive transactions`.
- Sharpen your skills with `MCQs` to prepare for competitive exams.

3. Simple Interest

- Master the `formula` for calculating simple interest.
- Differentiate between `principal`, `interest rate`, and `time period`.
- Solve `case-based problems` related to borrowing and lending.
- Practice `MCQs` for thorough preparation

4. Compound Interest

- Understand the `growth` of investments and savings.
- Differentiate between `simple interest` and `compound interest`.
- Solve problems with `annual`, `semi-annual`, and `quarterly` compounding.
- Practice `MCQs` for preparation.

5. Ratio and Proportion

- Grasp the basics of `ratios`.
- Solve problems on `proportional relationships`.
- Analyze scenarios involving `scaling`, `sharing`, and `dividing` quantities.

- Practice `MCQs` for preparation.

Number Related Topics

1. Number System

- Understand the classification of `natural numbers`, `whole numbers`, `integers`, `rational numbers`, and `irrational numbers`.
- Master `divisibility rules`, `factors`, `multiples`, and `place value`.
- Practice `MCQs` to improve understanding and problem-solving speed.

2. HCF and LCM

- Learn techniques to find `HCF` and `LCM`.
- Understand their applications in `scheduling` and `resource sharing`.
- Solve word problems involving `time`, `distance`, and `recurring patterns`.
- Practice `MCQs` for competitive exam preparation.

3. Average

- Understand `averages` and their significance.
- Solve problems on `weighted averages`, `missing numbers`, and `group data`.
- Apply averages in `performance analysis` and `time management`.
- Practice `MCQs` to enhance speed and accuracy.

Speed Work and Time Related Topics

1. Work and Time

- Understand the relationship between `work`, `time`, and `efficiency`.
- Solve problems involving `individuals` or `groups` working together.
- Analyze scenarios like `alternating work schedules` and `work completion rates`.
- Practice `MCQs` problems.

2. Pipes and Cisterns

- Understand the analogy between `pipes` and `work-time`.
- Solve problems with `multiple pipes` working together or alternately.
- Address challenges like `leaks` or partial closure.
- Practice `MCQs` to improve your skills.

3. Speed, Distance, and Time

- Master the formula: `Speed = Distance / Time`.
- Solve problems on `relative speed`, `average speed`, and `varying speeds`.
- Practice `MCQs` questions.

4. Problems on Trains

- Calculate the time for a train to cross `poles`, `platforms`, or other trains.
- Apply `relative speed` in train-related problems.
- Solve problems with trains of different `lengths` and `speeds`.
- Practice `MCQs` questions.

5. Boats and Streams

- Understand the impact of `stream direction` (upstream, downstream) on speed.
- Solve problems on `relative speed` and `effective speed` in flowing water.
- Analyze scenarios like `rowing competitions` or `river crossings`.
- Practice `MCQs` to test your understanding.

Probability and Combinations

1. Permutations and Combinations

- Understand the difference between `permutations` (arrangement) and `combinations` (selection).
- Learn key `formulas` and techniques for calculating arrangements and selections.
- Solve problems with `factorials`, `repetition`, and `circular permutations`.
- Practice `MCQs` to improve problem-solving skills.

2. Probability

- Understand `probability` as a measure of likelihood.
- Learn `formulas` for calculating probability in events.
- Practice `MCQs` to improve proficiency.

Progressions

1. Arithmetic Progression (AP)

- Understand `Arithmetic Progression` with a constant difference.
- Derive formulas for `general term (an)` and `sum of n terms (Sn)`.
- Apply `AP` in real-life problem solving.
- Solve problems on `missing terms`, `specific terms`, and `sum of series`.
- Practice `MCQs` and concept-based questions.

2. Geometric Progression (GP)

- Understand `Geometric Progression` with a constant ratio.
- Solve problems on `missing terms`, `specific terms`, and `sum of series`.

Miscellaneous Topics

1. Calendar

- Understand `days`, `months`, `leap years`, and `century years`.
- Learn `Odd Days` concept and calculation for day of the week.
- Use key `formulas` to find the day for any given date.
- Solve problems on `repeating calendar years` and calendar-based tricks.
- Practice `MCQs` and scenario-based questions.

2. Clocks

- Understand `clock structure`, `minute hand`, `hour hand`, and their movements.
- Solve `angle problems` between clock hands.
- Solve problems on `overlaps`, `right angles`, and `opposite directions`.
- Practice `clock puzzles` and time calculation problems.
- Practice `MCQs` and puzzle-based questions.

Logical Reasoning

1. Direction Sense

- Understand `directions` (North, South, East, West) and final direction after movements.
- Track `movements` and `turns` (right/left) to find final position.
- Solve problems with `multiple directions` and movement patterns.
- Practice `MCQs` for speed and accuracy.

2. Blood Relation

- Identify relationships like `father`, `mother`, `brother`, `sister`.
- Analyze clues to trace `family connections`.
- Solve problems with `family trees` and complex relationships.
- Practice `MCQs` to improve deduction skills.

3. Syllogism

- Understand `logical reasoning` and conclusion deduction.

- Break down `premises` to check conclusions.
- Work with `All`, `Some`, `No` premises.
- Solve `MCQs` to identify valid/invalid conclusions.

4. Arrangements

- Learn to arrange `people` or `objects` based on conditions.
- Apply `constraints` like sitting together or specific positions.
- Solve problems with multiple `arrangement conditions`.
- Practice `MCQs` to strengthen understanding.

5. Series

- Understand `number sequences` and identify next terms.
- Recognize patterns like `arithmetic progressions`, `geometric progressions`.
- Solve problems with varying `series types` and difficulty.
- Practice `MCQs` to improve pattern recognition.

Verbal Reasoning

1. Sentence Ordering

- Practice `MCQs` to improve sentence `ordering` skills.

2. Error Identification

- Practice `MCQs` to sharpen error `spotting` and correction.

3. Sentence Improvement

- Practice `MCQs` to improve sentence `quality`.

Episode 9 - System Design

Low Level Design

1. LLD Foundations

- What is `Low Level Design` (code-level system breakdown).
- Understanding `OOPS` (classes, objects, inheritance, polymorphism).
- How to think in `class relationships` and object interactions.
- Basics of `UML Diagrams` (class diagrams, sequence diagrams).
- How to approach `LLD interviews` step-by-step.

2. Design Principles

- `SOLID Principles` (core of maintainable code).
- `DRY` – avoid duplication.
- Introduction to `Design Patterns`
- Writing `extensible and scalable code`.

3. Creational Design Patterns

- `Singleton` – one instance control.
- `Factory Method` – object creation abstraction.
- `Abstract Factory` – families of related objects.
- `Builder` – step-by-step object creation.
- `Prototype` – cloning objects efficiently.

4. Structural Design Patterns

- `Adapter` – compatibility between interfaces.
- `Facade` – simplified interface over complex system.

- `Proxy` – controlled access.
- `Decorator` – dynamic behavior extension.
- `Composite` – tree-like object structures.

5. Behavioral Design Patterns

- `Observer` – event-based updates.
- `Strategy` – interchangeable logic.
- `Command` – encapsulate actions.
- `Iterator` – sequential access.
- `State` – behavior changes with state.
- `Template Method` – define skeleton logic.
- `Chain of Responsibility` – request flow handling.
- `Mediator` – centralized communication.

6. LLD Problem Solving (Machine Coding)

- Designing `Parking Lot System`.
- Designing `Vending Machine`.
- Designing `BookMyShow`.
- Designing `Splitwise`.
- Designing `Uber / Ola`.
- Designing `Elevator System`.

High Level Design (HLD)

1. System Design Foundations

- What is `High Level Design` (system architecture overview).
- Understanding `functional vs non-functional requirements`.
- How to break systems into `components & services`.
- `Capacity estimation` basics (users, traffic, storage).
- Step-by-step `system design interview approach`.

  note : HLD focuses on architecture, not code

2. Communication & Data Layer

- Basics of `Networking` (HTTP, TCP, latency).
- Designing clean `APIs` (REST principles).
- `SQL vs NoSQL` – when to use what.
- `Database scaling` (replication, sharding basics).
- Managing `data consistency vs performance`.

3. Core Infrastructure

- `Caching` strategies (Redis, in-memory).
- `CDN` and content delivery optimization.
- `Load Balancing` (horizontal scaling).
- `API Gateway` role in distributed systems.
- `Message Queues` (async communication).
- `Microservices Architecture` fundamentals

4. Advanced Distributed Systems

- `Reliability` and system availability.
- `Fault Tolerance` and failure handling.
- `Observability` (logs, metrics, tracing).
- `Rate Limiting` and traffic control.
- `Authentication & Authorization`.
- Designing `Search Systems`.

5. Real World System Design Problems

- `12 real-world system` design problems.